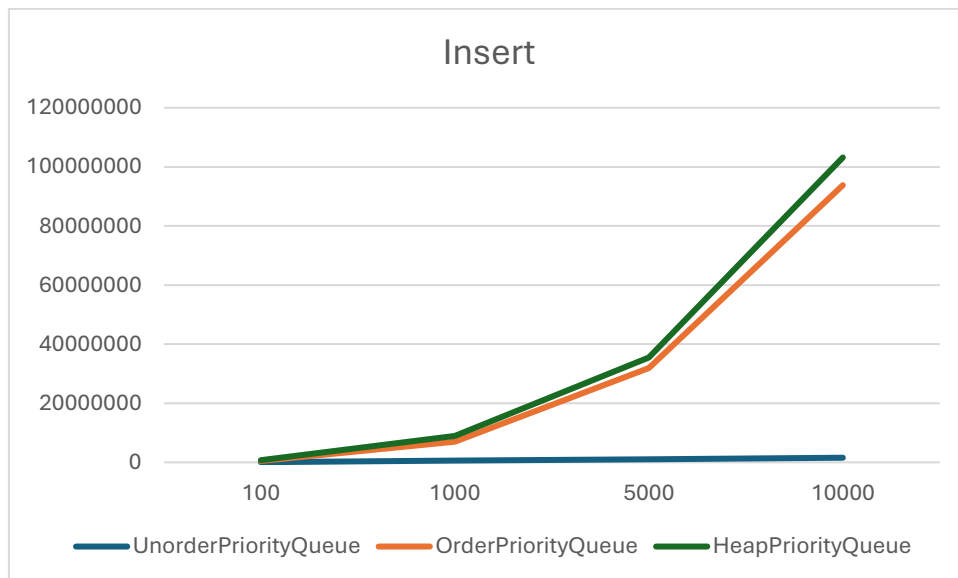


Priority Queue

Comparación de los Priority Queue respecto al insert:

Tiempos/Elementos	Insert		
	UnorderPriorityQueue	OrderPriorityQueue	HeapPriorityQueue
100	58400	471500	739100
1000	647600	6952000	8947800
5000	1007300	31863500	35509300
10000	1567000	93795300	103181300



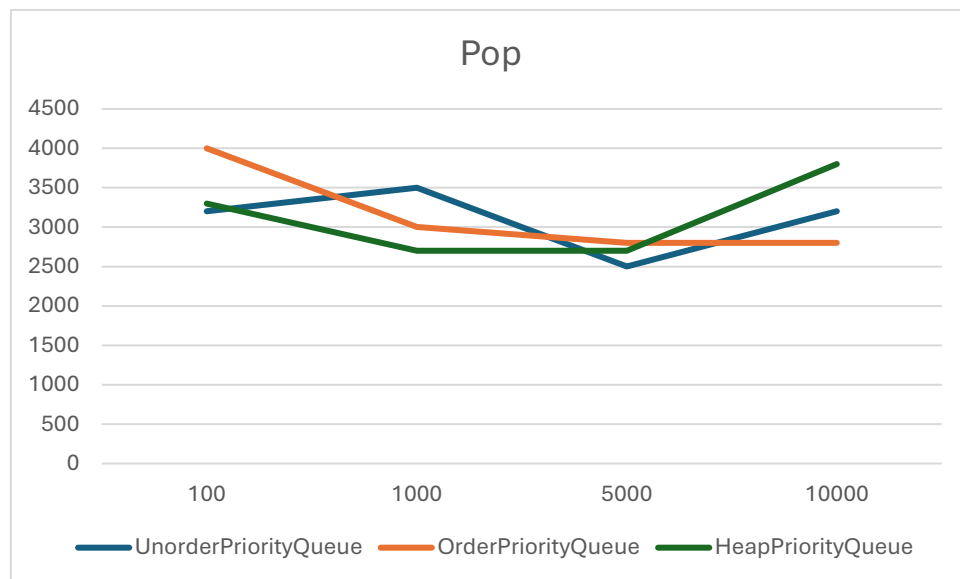
Se utilizan datos random, que tienen como máximo la capacidad de la queue.

Respecto al campo teórico, el que mas se adapta el UnorderPriorityQueue, ya que este posee un orden de complejidad constante ($O(1)$), además el OrderPriorityQueue posee su complejidad N ($O(N)$), y por ultimo el Heap cumple con la complejidad logarítmica ($O(\log(N))$).

Entonces comparando el orden empírico y el esperado, se relación mucho ambos y coinciden bastante a como debería ser.

Comparaciones de los PriorityQueue respecto al pop:

Tiempos/Elementos	Pop		
	UnorderPriorityQueue	OrderPriorityQueue	HeapPriorityQueue
100	3200	4000	3300
1000	3500	3000	2700
5000	2500	2800	2700
10000	3200	2800	3800



Se utilizan datos random, que tienen como máximo la capacidad de la queue.

Respecto al campo teórico en el Order debería ser constante, en el Unorder debería ser de complejidad lineal ($O(N)$), y por ultimo el Heap debería ser de complejidad logarítmica ($O(\log N)$), para comparar entre los datos empíricos y los teóricos el que mas similitud posee es el orden del OrderPriorityQueue, ya que se ve como es un pico al inicio y luego se mantiene semi constante y en descenso